

Documentacion de practicas Laravel

Introduccion

En el siguiente documento iremos viendo las diferentes practicas realizadas del dossier de Laravel. En cada ejercicio pongo el enunciado y la ruta del web.php para no estar copiando al completo el Controlador y la vista, pero estos se encuentran rápido en el "proyectoTrabajar", controladores en app/Http/Controllers y vistas en resources/views.

Para subir el proyecto al zip, tuve que borrar la carpeta de vendor, y así reducirle el peso.

Para restaurar:

```
$ composer install  
  
$ php artisan key:generate
```

Índice:

- [Practicas 1 - 5](#)
- [Practicas 5 - 10](#)
- [Practicas 10 - 15](#)
- [Practicas 15 - 20](#)

Practica 1

Modificar el fichero web.php para que las peticiones GET (parecido al ejemplo anterior) al raíz de la aplicación: "/" muestren un mensaje que diga: "Under construction"

```
Route::get('/', function () {echo "Under Construction";});
```

Practica 2

Modificar el fichero web.php para que las peticiones POST a: /pruebita muestren el mensaje: "se ha ejecutado una petición POST a la dirección: /pruebita "

Probar a hacer la petición POST ¿ muestra lo solicitado ?

Si

¿ qué ocurre si se hace mediante una petición GET ?

Da un error

Volver a reestablecer la protección CSRF y hacer de nuevo la petición POST ¿ qué muestra ahora ?

Un error 419 unknow status

Nota: Desde Rested se puede enviar la petición POST, así como Postman o cualquier otra utilidad. Incluso desde el inspector de Firefox permite modificar una petición GET a POST y reenviar. En el inspector de firefox al editar la petición y pasar a post te da la respuesta en el "preview" no en la página renderizada del navegador

```
Route::post('/pruebita', function () {echo "se ha ejecutado una petición POST a la dirección: /pruebita";});
```

Practica 3

Crear una ruta para TODA petición (ya sea GET, POST, ...) hacia /relatos/numero (recordar que hemos visto una opción para recoger todo tipo de petición) De tal forma que numero deba ser un número y muestre el mensaje: "petición recibida para el parámetro: numero"

```
Route::any('/relatos/{numero}', function ($numero) {echo "petición recibida para el parámetro: $numero";})->where('numero', '[0-9]+');
```

Practica 4

Crear una ruta para el raíz: "/" En una primera implementación mostrará el mensaje: "página raíz de nuestra aplicación" que se resolverá en el propio web.php Haremos una segunda versión de esta actividad en la que redireccionará hacia el controlador y la función pertinente y allí se mostrará un mensaje que indique adicionalmente que se ha respondido desde el controlador

Practica 5

Crear un controlador llamado: ListarProductos que sea redireccionado en web.php cuando se acceda al raíz: "/" y muestre un mensaje que diga: "Ejecutando el controlador ListarProductos mediante get". (si la llamada fue get. En el caso de que la llamada fuera post deberá decirlo)

```
Route::get('/listarProductosGet', [Controller_Pr05_ListarProductos::class, 'listarProductosGet']);

Route::post('/listarProductosPost', [Controller_Pr05_ListarProductos::class, 'listarProductosPost']);
```

Practica 6

Comprobar que la anotación @var para un objeto permite que el ide con inteliphense nos ayude con los atributos y los métodos

Practica 7

Reproducir la vista descrita. Crear una tabla html por cada primo con un encabezado en la tabla que nos diga que campo estamos visualizando.

```
Route::get('/numPrimos', [Controller_Pr07::class,'numPrimos']);
```

Practica 8

Agregar al comienzo de la vista el mensaje(sustituye por la hora/día actual): Son las: 17:53 del día: 29-11-2020
Nota: buscar información y usar la función PHP date()

Se realiza en la vista y controlador de la practica 7

Practica 9

El comando sleep() en php permite pausar la ejecución la cantidad de segundo especificada como parámetro. Modificar el ejemplo anterior para que lo muestre 3 veces con una espera de 1 segundo entre una iteración y la siguiente, mostrando de forma actualizada la información de los segundos desde 1970

```
Route::get('/time', function(){  
    return view('Pr09_time');  
});
```

Practica 10

Generar una lista de números aleatorios de 0 a 100 en el controlador. Desde nuestra plantilla blade mostraremos primero la lista de números obtenidos menores de 50 y un poco más abajo en la página los mayores que 50. Hacer uso de las directivas @if para que al mostrar aquellos que sean mayores de 50

```
Route::get('/numAleatorios', [Controller_Pr10::class,'numerosAleatorios100']);
```

Practica 11

Enviar en un textarea una lista de palabras separadas por comas. Mostrar en una lista html esas palabras recibidas (una palabra por cada

- de la lista) convertidas todas a mayúsculas. Para ello se usará el bucle: @for (cuidado! No el foreach)
- Observar que eso implicará "contar" el número de elementos que tiene la colección de palabras

```
Route::get('/listaPalabras', [Controller_Pr11::class,'listaPalabras']);
```

Practica 12

Ubicar imágenes en la carpeta descrita para las imágenes que quieras mostrar (mínimo 5). Hacer que se visualicen en el navegador las imágenes en nuestra vista nota: Poner el renderizado que se considere más apropiado

```
Route::get('/imagenes', [Controller_Pr12::class, 'mostrarImagenes']);
```

Practica 13

Crear un formulario que envíe nombres de colores en cada ejecución del usuario. Obtendrá por respuesta una página con la lista de colores que ha ido introduciendo (usar session() para almacenar la lista de colores) (es un formulario post tendremos que tener en cuenta @csrf leer más abajo)

```
Route::get('/mostrarColores', [Controller_Pr13::class, 'mostrarColores']);

Route::get('/storeColores', [Controller_Pr13::class, 'storeColores']);
```

Practica 14

Una forma fácil de visualizar el token csrf es mediante: {{ csrf_token() }} Introducir en la práctica 12 ese código y comprobar que está activo.

Si lo está

Practica 15

Crear un formulario POST Con los datos de un posible usuario (nombre, edad, gustos, etc) En cada ejecución de este formulario se le muestra al usuario la información almacenada del usuario en session() Observar que si se envía el formulario sin rellenar algún campo, se mantendrá la información anterior respecto a ese campo

```
Route::get('/formularioUsuario', function(){
    return view('Pr15_formularioUsuario');
});
Route::get('/almacenarInfo', [Controller_Pr15::class, 'almacenarInfo']);
```

Practica 16

Crear un fichero con nombre y dirección de correo por fila (en formato csv) almacenado en Storage Leer el fichero y mostrarlo en pantalla

```
Route::get('/leerFichero', [Controller_Pr16::class, 'leerFichero']);
```

Practica 17

Crear un formulario que se introduzca un nombre y cree un directorio en storage con ese nombre

```
Route::get('/crearDirectorio', function(){
    return view('Pr17_formCrearDirectorio');
});

Route::get('/procesarCrearDirectorio', [Controller_Pr17::class, 'crearDir']);
```

Practica 18

Crear un fichero con nombre y dirección de correo por fila (en formato csv) almacenado en Storage Leer el fichero y mostrarlo en pantalla

```
Route::get('/crearFichero', function(){
    return view('Pr18_formCrearFichero');
});

Route::any('/subirFile', [Controller_Pr18::class, 'crearFichero']);
```

Practica 19

Mostrar en una página una lista de ficheros de una carpeta en storage. Cuando se pulse en el nombre del fichero se descargará

```
Route::any('/cargarFilesDownload', [Controller_Pr19::class, 'cargarFiles']);

Route::any('/downloadFile', [Controller_Pr19::class, 'descargarFiles']);
```

Practica 20

Continuando con el ejemplo anterior crear una opción para borrar los ficheros listados

```
Route::any('/cargarFilesDelete', [Controller_Pr20::class, 'cargarFiles']);

Route::any('/deleteFile', [Controller_Pr20::class, 'descargarFiles']);
```